GUÍA DE DEFENSA — PROYECTO INTEGRADOR

Visión Estereoscópica con ESP32-CAM, AR y Filtro de Kalman

Michael Lata · Jordy Romero | Visión Artificial G8 | UPS

ARQUITECTURA MVC C++17

Roles

MODELO — solo algoritmos, nunca dibuja
`StereoCalibration` · `StereoProcessor` · `KalmanDepth` · `CameraSource`

VISTA — solo dibuja, nunca procesa
`DisplayManager` · `ARRenderer`

CONTROLADOR — bucle principal
`StereoController::run()`

Por qué MVC: cambiar cámara (HTTP→USB) no toca el procesador. Cambiar display no toca SGBM. Patrón **Strategy** en `CameraSource`.

ESP32-CAM — CAPTURA HTTP

IPs: `172.20.10.2` (der) · `172.20.10.3` (izq). Endpoint `/capture` entrega un JPEG por petición.

Hilo productor — workerLoop()

```
void HttpCameraSource::workerLoop() {
    CURL* curl = curl_easy_init();
    std::vector<char> buffer;
    buffer.reserve(32 * 1024); // 32 KB sin realocar
    curl_easy_setopt(curl, CURLOPT_NOSIGNAL, 1L); //
        multi-hilo
    while (running_.load()) {
        buffer.clear();
        curl_easy_perform(curl); // descarga JPEG
        // Decodificar en RAM, sin disco
        cv::Mat raw(1, buffer.size(), CV_8UC1, buffer.
            data());
        cv::Mat frame = cv::imdecode(raw, cv::
            IMREAD_COLOR);
        std::lock_guard<std::mutex> lock(mutex_);
        latestFrame_ = std::move(frame); // O(1), sin
            copiar
        hasFrame_ = true;
    }
}
```

Pregunta: por que un hilo por camara?

El bucle principal no se bloquea esperando la red. Si una camara tarda, la otra sigue corriendo. `read()` retorna el frame mas reciente sin esperar.

PIPELINE ESTÉREO — 7 PASOS

`StereoProcessor::process(rawLeft, rawRight, result)`

1. **Rectificación:** `cv::remap()` con mapas precalculados
2. **Escala:** ajusta si frame difiere de calibración
3. **Grises:** `cvtColor(BGR→GRAY)`
4. **CLAHE:** mejora contraste local
5. **SGBM:** calcula disparidad (Q16.4)
6. **WLS:** rellena huecos, suaviza bordes
7. **Reproyección 3D:** `reprojectImageTo3D(Q)` → metros

CALIBRACIÓN — MATRIZ Q

Qué almacena `StereoCalibration`:

- `map1x, map1y` — rectificación cámara izquierda
- `map2x, map2y` — rectificación cámara derecha
- `Q` — matriz 4×4 de reproyección

Forma explícita de Q:

$$\mathbf{Q} = \begin{pmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & -\frac{1}{B} & \frac{c_x - c'_x}{B} \end{pmatrix}$$

$c_x = 170.3\,\text{px},\ f = 328.26\,\text{px},\ B = 66.8\,\text{mm}$

Profundidad métrica:

$$Z = \frac{f \cdot B}{d} = \frac{328.26 \times 0.0668}{d\,[\text{px}]}$$

A 30 cm: $d \approx 73$ px A 90 cm: $d \approx 24$ px

A mayor distancia, un pixel de error en d produce mayor error en Z.

CLAHE — MATEMÁTICA COMPLETA

La ecualización estándar mapea intensidades con la CDF:

$$s_k = (L-1)\sum_{j=0}^{k}\frac{n_j}{M\times N}$$

El problema: zonas uniformes tienen pendientes gigantes de CDF que amplifican ruido. CLAHE limita el histograma en cada tesela 8×8 px antes de integrarlo:

$$h'(r) = \min\Big(h(r),\ \text{clipLimit}\times\frac{M_t\times N_t}{L}\Big)$$

Con `clipLimit=2.0`: la pendiente máxima queda acotada a 2, duplica el contraste local sin amplificar ruido. El exceso se redistribuye uniformemente.

SGBM — FUNCIÓN DE COSTO

$$E(\mathbf{D}) = \sum_p \Big[C(p,D_p) + \sum_{q\in\mathcal{N}_p}\big(P_1\mathbf{1}_{|D_p-D_q|=1} + P_2\mathbf{1}_{|D_p-D_q|>1}\big)\Big]$$

- $C(p,D_p)$ — costo de matching en bloque $b\times b$
- $P_1 = 8\cdot 3\cdot b^2$ — penaliza cambios suaves (1 px)
- $P_2 = 32\cdot 3\cdot b^2$ — penaliza saltos bruscos ($P_2 = 4P_1$)
- `MODE_SGBM_3WAY`: 3 direcciones de escaneo

Por qué numDisp múltiplo de 16: restricción interna de OpenCV para alineación SIMD. Se normaliza: `((n+15)/16)*16`.

WLS — FILTRO DE MÍNIMOS CUADRADOS

$$E_{\text{WLS}}(u) = \sum_i\Big[w_i(u_i-d_i)^2 + \lambda\sum_{j\in\mathcal{N}_i}\frac{(u_i-u_j)^2}{e^{-|I_i-I_j|/\sigma_c}+\varepsilon}\Big]$$

- d_i — disparidad cruda de SGBM
- I_i — píxel de la imagen guía (color)
- $\lambda = 8000$ — fuerza del suavizado global
- $\sigma_c = 1.5$ — sensibilidad a bordes de color

Necesita disparidad R→L para detectar píxeles ocluidos:

```
rightMatcher_ = ximgproc::createRightMatcher(sgbm_);
rightMatcher_->compute(grayR, grayL, disp16R); // R->L
wlsFilter_->filter(disp16L, leftRect,
                   filtered, disp16R);
```

FILTRO DE KALMAN — MATEMÁTICA COMPLETA

Ecuaciones de Kalman 1D

Predicción: $\hat{x}^- = \hat{x} \quad P^- = P + Q$

Ganancia: $K = \dfrac{P^-}{P^- + R}$

Corrección: $\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$

Covarianza: $P = (1-K)\,P^-$

Estado estacionario — ecuación de Riccati escalar:

$$\bar{P} = \bar{P} + q - \frac{\bar{P}^2}{\bar{P}+r} \Rightarrow \bar{P} = \frac{q+\sqrt{q^2+4qr}}{2}$$

Con $q = 0.001$, $r = 0.01$: $\bar{P} \approx 0.00291\,\text{m}^2$

Ganancia de estado estacionario:

$$K_\infty = \frac{\bar{P}}{\bar{P}+r} = \frac{0.00291}{0.01291} \approx 0.225$$

Cada frame: 22.5% nueva medición + 77.5% predicción previa.

Subir Q (trackbar): K_∞ sube → sigue rápido pero más ruidoso.

Subir R (trackbar): K_∞ baja → muy suave pero lento.

```
double KalmanDepth::update(double measurement) {
    covariance_ += q_;          // prediccion: crece
    const double K =
        covariance_ / (covariance_ + r_);
    estimate_   += K * (measurement - estimate_);
    covariance_  = (1.0 - K) * covariance_;
    return estimate_;
}
```

MEDIANA vs. MEDIA — `medianDepthAt`

$$Z_{\text{median}} = \text{sort}(\{Z_i\})[\lfloor n/2\rfloor] \quad \text{vs.} \quad Z_{\text{mean}} = \frac{1}{n}\sum Z_i$$

Si un pixel tiene $Z = 9.999\,\text{m}$ (outlier), la media se desplaza $\approx 0.08\,\text{m}$. La mediana lo ignora. Implementación $O(n)$ sin ordenar todo:

```
// nth_element: garantiza que [mid] sea correcto en O(n)
// sin ordenar todo el vector (que seria O(n log n))
std::nth_element(zVals.begin(),
                 zVals.begin() + mid,
                 zVals.end());
return zVals[mid];
```

AR — MATEMÁTICA DEL EFECTO

Capa 1 — Distorsión sinusoidal

Amplitud con decaimiento cúbico:

$$A(Z) = \text{clamp}\left(\frac{0.6}{Z^3},\ 0,\ 80\right) \text{ [px]}$$

A 20 cm: $A \approx 75$ px A 50 cm: $A \approx 5$ px A 80 cm: $A < 2$ px

Desplazamiento animado de cada píxel:

$$\delta x = A\sin(0.1y + \omega t), \quad \delta y = A\cos(0.1x + 0.75\omega t)$$

$$\omega(Z) = \frac{12}{Z} \text{ [rad/s]}$$

Manipulación directa de píxeles (exigencia de rúbrica):

```
cv::Vec3b* dst = image.ptr<cv::Vec3b>(y);
const cv::Vec3f* dep = points3D.ptr<cv::Vec3f>(y);
for (int x = 0; x < cols; ++x) {
    const float Z = dep[x][2];
    float ox = A * sin(y*0.10f + t*speed);
    float oy = A * cos(x*0.10f + t*speed*0.75f);
    dst[x] = src.at<Vec3b>(y+oy, x+ox); // pixel
        desplazado
}
```

Capa 2 — Aura pulsante

Radio pulsante a 3 Hz:

$$r(t) = r_{\text{base}} \cdot (0.9 + 0.1\sin(3t))$$

Alpha (nunca tapa la imagen):

$$\alpha = 0.35 + 0.25 \cdot \min\left(1, \frac{20}{Z_{\text{cm}}}\right) \in [0.35,\ 0.60]$$

Destellos orbitales que se aceleran al acercarse:

$$\theta_i(t) = t \cdot v_{\text{spin}} + \frac{2\pi i}{n}, \quad v_{\text{spin}} = 1 + \frac{2 \cdot 20}{Z_{\text{cm}}}$$

FUNCIONES AVANZADAS NO VISTAS EN CLASE

std::thread + mutex + atomic

```
std::atomic<bool> running_{true}; // stop sin race
std::mutex mutex_;                // exclusion mutua
std::lock_guard<std::mutex> lock(mutex_); // RAII
// Destructor: parada limpia
running_.store(false);
if (worker_.joinable()) worker_.join();
```

atomic: escritura visible entre hilos sin mutex. *lock_guard:* libera el mutex automaticamente.

cv::ximgproc — modulo contrib

No es `opencv2/opencv.hpp` estandar. Requiere `opencv_contrib`. Necesita disparidad L→R y R→L para detectar oclusiones.

reinterpret_cast — callback de raton

```
// OpenCV exige callback C estatico (sin 'this')
// Se pasa el objeto como void* y se recupera:
auto* self = reinterpret_cast<StereoController*>(
    userdata);
```

cv::getTickCount() — animacion

```
// Reloj alta precision, independiente de plataforma
const double t = (double)cv::getTickCount()
               / cv::getTickFrequency(); // segundos
```

CURLOPT_NOSIGNAL — multi-hilo seguro

libcurl usa `SIGALRM` por defecto para timeouts. Las senales son globales: crashean en multi-hilo. `NOSIGNAL=1` usa `select/poll` internos en su lugar.

std::move — sin copiar frames

```
// Sin move: copia 921 600 bytes (640x480x3)
latestFrame_ = frame;
// Con move: transfiere punteros en O(1)
latestFrame_ = std::move(frame);
```

TRACKBARS — FLUJO DE DATOS

Exigencia de rubrica cumplida

"Controles para ajuste fino del SGBM, filtros de ruido y estabilizacion de la medicion." Los 10 trackbars cubren todo.

Slider → `tb_*_` $\xrightarrow{\texttt{readConfig()}}$ ProcessorConfig $\xrightarrow{\Delta \text{ detectado}}$ re

Por que **int escalado** y no float: OpenCV `createTrackbar` solo acepta `int*`.

Trackbar	Escala	Rango real
numDisp	$\times 16$	16–256 px
blockSize	$\times 2 + 3$	3–21 px
CLAHE clip	$\div 10$	0.1–8.0
WLS lambda	$\times 1000$	1k–20k
WLS sigma	$\div 10$	0.5–3.0
Kalman Q	$\times 10^{-4}$	10^{-4}–10^{-2}
Kalman R	$\times 10^{-3}$	10^{-3}–10^{-1}

Titulo dinamico (valores reales cada frame):

```
cv::setWindowTitle(WIN_CTRL, buf);
// "numDisp=192px block=5px | CLAHE=2.0 | ..."
```

Q&A — DEFENSA

Preguntas frecuentes

Por que mediana y no promedio?
Un outlier Z=9999 no mueve la mediana. Si mueve la media.

Por que numDisp multiplo de 16?
Alineacion SIMD interna de OpenCV.

Por que rectificar antes de SGBM?
SGBM solo busca en 1D (horizontal). Sin rectificacion seria 2D.

Por que escalar Q de QVGA a VGA?
Calibracion en 320x240, runtime en 640x480. Factor 2 en c_x, c_y, f.

Por que MODE_SGBM_3WAY?
Menos artefactos de rayas que el modo 1 camino. Mejor que 8 caminos en velocidad.

Por que P2 = 4*P1?
Recomendacion Hirschmuller (2008): saltos bruscos de disparidad muy costosos, superficies coherentes.

Que es la baseline?
Distancia fisica entre camaras: 66.8 mm. Mayor = mas precision en objetos lejanos.

Por que dos filtros Kalman?
`kalmanCenter_`: siempre activo para AR.
`kalmanClick_`: solo al hacer clic, se resetea al cambiar punto.

Que demuestra el efecto AR?
Manipulacion directa de pixeles con `ptr<Vec3b>`. Reacciona a Z medido. Matematica propia (decaimiento cubico, ondas).

Por que CURLOPT_NOSIGNAL?
libcurl usa SIGALRM por defecto, que crashea en multi-hilo. NOSIGNAL=1 usa select/poll internos, seguro con std::thread.

Comando: `./stereo_measure ../config/stereo_calib.yml http://172.20.10.2/capture http://172.20.10.3/capture`
Teclas: Q/ESC = salir R = log CSV C = reset Kalman Click = medir punto